PQMigrate: Evidence-Driven and Abstention-Aware Planning for Post-Quantum Cryptographic Migration

Dr. Deepak Uplaonkar, Ratnesh Patnaik, Saravanan Kumar, Vaibhavi Shah, and Prince Ostwal

Abstract. PQMigrate is a bounded source-analysis and advisory-planning prototype. Its completed quantitative study is a single-author synthetic pilot of 72 Python RSA snippets, not an independent real-code evaluation. The historical results describe an authored mixture of supported operations, unsupported operations, negative controls and malformed inputs. They establish neither generalization nor migration safety. This revision separates source-level operations from application roles, relates the implementation to cryptographic inventory and API-misuse analysis, and specifies an independent evaluation protocol. Real-code annotation and the external Semgrep comparison remain ongoing and unscored. Result tables are retained unchanged. Planner recommendation correctness, Go accuracy, deployed interoperability and PQC transformations have not been evaluated by this pilot.

Index terms: post-quantum cryptography, cryptographic inventory, static analysis, role inference, selective prediction, reproducibility.

I. Introduction

NIST FIPS 203 defines ML-KEM for key encapsulation; FIPS 204 defines ML-DSA for signatures [1], [2]. FIPS 205 defines the hash-based signature standard SLH-DSA [5]. These functions are distinct. Replacing an RSA signature interface and redesigning an RSA key-transport interface require different evidence about key formats, peers, message sizes and failure handling. A KEM establishes a shared secret and does not offer arbitrary-message encryption as a drop-in replacement [1]. Finding an algorithm name is consequently an inventory lead, not a complete migration decision.

Cryptographic-agility work treats dependency management and coordinated change as research problems [3], [4]. NIST's NCCoE migration project separates cryptographic discovery from interoperability work [8]. PQMigrate addresses part of the evidence-management problem: observations link to source locations, bounded inference attempts to resolve their use, and an advisory planner records conditions and blockers. It does not establish that suggested replacements will interoperate or be secure in the target application.

Standards and deployment status must be dated. At the verification date, 21 October 2026, NIST lists Falcon and HQC standardization as ongoing [6]. RFC 9964 specifies ML-DSA identifiers for JOSE and COSE [7]. This protocol specification is not evidence that a particular dependency, peer or deployment supports those identifiers. No JOSE/PQC integration or transformation is evaluated here.

II. Contribution and evidence boundary

The contribution is an inspectable prototype, an evidence-linked representation of observations and conditional advice, and a diagnostic pilot that exposes inference limits. Python and Go parsing support is an implementation capability; the completed numerical pilot evaluates only synthetic Python RSA snippets. A source-level invocation is not proof that a production execution reached it. Test files may contain real crypto invocations, while an import, comment or algorithm descriptor alone is not an operation.

The paper asks whether bounded evidence can distinguish inventory leads, operations and application roles, and how unresolved evidence should be reported. It does not answer whether the prototype is accurate across repositories or whether its advice reduces migration risk. The original result pages are reproduced verbatim as pages 4-6, including their tables, qualifications and historical Draft 3 headers. Newly verified references and the revised protocol appear around those preserved pages. No new numerical result has been substituted.

III. Related work

Discovery and migration dependencies. Cryptoscope describes compiler-based analysis using control and data flow to identify cryptographic usage [9]. Hasan et al. model security dependencies and present migration case studies [4]. These works motivate connecting inventory entries to use and dependencies. PQMigrate's bounded heuristics are not equivalent to whole-program compiler analysis, and this pilot provides no shared-corpus comparison or performance ranking against these systems.

Cryptographic API correctness. CogniCrypt combines assistance for generating cryptographic code with checking its integration [10]. CrySL specifies correct API usage and supports context- and flow-sensitive checking [11]. CryptoGuard uses program-slicing refinements for detecting cryptographic vulnerabilities in Java projects [12]. CryptoAPI-Bench supplies Java API-misuse cases [13]. These tasks concern usage constraints or weaknesses. PQMigrate instead attempts to identify migration-relevant operation roles and produce conditional advice. This distinction does not make it a misuse detector, and none of the cited Java results transfers to the Python/Go setting without measurement.

Inventory tools and interchange. CodeQL documents a Python query for broken or weak cryptographic algorithms [14]. The Sonar cryptography plugin and CBOMkit provide cryptographic discovery and CBOM tooling [15], [16]. CycloneDX CBOM represents cryptographic assets and relationships [17]; SARIF standardizes static-analysis result interchange [18]. An export can satisfy a schema while containing an incorrect source interpretation. Structural validation is therefore separate from detection accuracy, role correctness and downstream ingestion. These tools are relevant comparators or integration targets, not baselines already evaluated in this manuscript.

Reject options and reporting. Chow studies the error/reject trade-off [20]; selective-classification research formalizes coverage and error among accepted predictions [19], [21]. This motivates reporting answered-case correctness together with coverage. PQMigrate does not satisfy the assumptions of a learned selective classifier or inherit a statistical guarantee from those papers. In particular, absence of a resolved role differs from a planner's decision to abstain from advice. The completed pilot measures the former, not the latter.

External rule-engine baseline. Semgrep runs published source patterns through its analysis engine [26]. The separate baseline implementation pins engine version, rules, source hashes and execution limits. Custom rules define the evaluated detector; results cannot be generalized to all Semgrep rule sets or commercial capabilities. Its execution ledger is not an accuracy evaluation. Native spans must be aligned to locked operation units before comparison.

IV. System model and inference

Discovery produces source-linked leads, inference attempts identity and role resolution, and the intermediate representation links findings to a versioned migration knowledge base. The planner exposes advice with conditions, evidence and blockers. JSON, SARIF and CBOM exports serialize observations; they are not additional evidence about the target's behavior [17], [18], [24]. Scanned source is treated as input rather than imported or executed.

For the inspected Python resolver, assignment collection tracks selected RSA key-generation calls and selected random-byte sources. A subsequent pass associates calls with tracked names in a bounded scope. This is name-based evidence, not a general points-to analysis. Imports, assignments, wrappers, mutation and branch conditions may invalidate the inferred association. A role assigned from such evidence remains a fallible inference [24].

An encryption of bytes from a tracked random source is a heuristic proxy used by the historical implementation for key transport. Randomness does not establish that those bytes are key material. The reviewer SOP instead requires evidence of their application use. The preserved diagnostic results expose this proxy's failure on non-key encryptions; the revision does not redefine those errors as correct labels.

V. Methodology and interpretation

Completed synthetic pilot. PQMigrateBench v0.1 contains 72 authored Python RSA snippets. One developer authored and labeled the corpus, including 52 operation snippets, 16 negative controls and four designated malformed inputs. Historical detection/role scores use the 68 parseable snippets, retaining unsupported valid operations in the operation denominator. The four malformed inputs remain a separate accounting category. This is a synthetic pilot with deliberately selected supported and unsupported families; it is not a random sample of real software [24]. The unchanged experimental definitions and result tables follow on pages 4-6.

The evaluated source revision is bcfb5e40fbc8eb74e1c9 19114f54031d03ab44e9 [24]. The corpus SHA-256 is recorded in the reproducibility audit. Source inspection establishes how metrics are calculated but does not independently verify every historical external gate log. The two clean-gate runs reported on the preserved pages remain historical reports; a public immutable log release was not available for this revision. New Semgrep runs must not be relabeled as those historical experiments.

Units and coupled outputs. The pilot's scoring unit is a snippet, whereas the real-code protocol uses source-level invocation units. A key may be generated, loaded, used to sign and serialized in one function: those invocations must not silently become one scored call. Historic coupled presence/known-role outputs must be distinguished from operation detection independent of role. An unresolved receiver is not a proven negative. A valid but unsupported operation remains eligible for annotation and failure accounting.

JWT and aggregation limits. Source inspection shows a literal RS-prefixed algorithm check for recognized JWT encoding. RFC 7518 defines PS256 as an RSA-PSS algorithm [22], so an RS-only check does not cover all RSA JWT signatures. The preserved supplementary probe is diagnostic, not an additional test-set estimate. Mixed roles, wrappers, cross-scope flow, aliasing, reassignment and dynamic behavior can defeat bounded inference. The implementation's ordering and aggregation choices are not a soundness proof [24].

Output masks. The session-secret and protocol variants in the pilot mask information in existing outputs. They demonstrate consequences for the reported fields; they are not independent implementations that disable analysis features. The old scaffold's shorthand "ablation" labels should be read in this limited sense. The numerical table remains untouched. A causal claim about a separately implemented analyzer would require a new experiment.

Mutation policy. The inspected policy checks role, construction support, interoperability evidence, tests and operator authorization, refusing mutation when requirements are false or unknown [24]. That control-flow property does not establish correct recommendations or secure transformed code. Tests, schema validity, source preservation, peer compatibility and cryptographic correctness are different obligations. Planner advice and unsafe-recommendation rates require separate independently adjudicated outcomes; this pilot supplies none.

Statistical scope. Perfect correctness among selected answered cases does not imply perfect reliability. Reject-option literature motivates joint coverage/error reporting but supplies no bound for this authored sample [19]-[21]. The zero-event discussion on the preserved pages is conditional on sampling assumptions; its citation [23] does not make correlated, purposively constructed snippets representative. No real-code precision, recall, TPR, agreement or confidence interval is asserted here.

The study's next stage is ongoing independent source annotation. Its revised sampling, calibration, taxonomy, locking and scoring requirements are specified after the unchanged pilot pages. Planned quantities are distinguished from observed results throughout.

VI. COMPLETED PILOT: EXPERIMENTAL DESIGN

A. Corpus and labels

PQMigrateBench v0.1 contains 72 single-author synthetic Python RSA cases (Table II). Detection and role metrics score 68 parseable cases: 52 labeled operations and 16 negative controls. Four designated malformed inputs are retained separately. Unsupported but syntactically valid operations remain in the 52-operation denominator; they are not removed because the resolver cannot handle them.

Table II. Completed pilot composition. Outcomes describe the resolver, not migration recommendations.

Category	Cases	Full resolver outcome
Direct signature	12	12 correct roles
JWT signature	8	8 correct roles and contexts
Key transport	12	12 correct roles
Asymmetric encryption	8	8 correct roles
Unsupported valid operations	12	12 unanswered, counted as misses
Negative controls	16	16 negative predictions
Malformed inputs	4	Separately accounted
Total	72	68 scored for detection/role

Nominal train, development, and test assignments contain 44, 14, and 14 cases. They are synthetic partitions, not independently collected repository holdouts. The resolver is not a learned model. Naming a partition “test” does not establish independence from author decisions or related scenario leakage. Labels have not undergone independent annotation or adjudication, and inter-rater agreement is unavailable.

Construction directly shapes the headline counts: the 12 unsupported cases were selected by the resolver's author, while the 40 supported cases and 16 negative controls exercise known scenario families. Their proportion determines observed coverage. These results are implementation diagnostics, not estimates of naturally occurring unsupported prevalence or out-of-sample accuracy.

The nominal partitions miss 8/32, 2/10, and 2/10 gold operations; their similar construction replicates the same limitations and supplies no independent validation.

B. Executed baselines and sensitivity checks

The lexical baseline flags an operation when a lowercased snippet contains `rsa`; otherwise it returns unknown. It does not resolve imports and always leaves role and operation labels unknown. It is an intentionally weak inventory heuristic, not a representative implementation of every lexical analyzer.

The AST baseline matches attribute calls named sign, verify, encrypt, or decrypt without resolving receiver identity. It returns a result only when observed method-derived roles are unique. Sign/verify maps to signature; encrypt/decrypt maps to encryption. It does not distinguish transported key material or recognize the JWT wrapper.

The full configuration calls the bounded resolver in Section V. Two post-prediction output masks rewrite key transport as encryption or protocol as unknown. They are metric-sensitivity checks, not component ablations; inference is not rerun with stages disabled. The derivative reporting artifact names them `session_secret_output_mask` and `protocol_context_output_mask`, retaining original historical identifiers solely as provenance. Frozen release evidence is not rewritten.

All configurations use identical case IDs, labels, assignments, and accounting classes. Malformed cases are excluded from detection/role denominators for every configuration, including the non-parsing lexical baseline. The harness does not implement uniform per-case timeout/crash measurement; no comparative robustness result is claimed. Regex-call and external-tool baselines are planned, not executed.

C. Metrics and denominators

On the 68 scored cases, detection uses true positives, false positives, false negatives, and true negatives:

$$P = TP/(TP + FP),\ R = TP/(TP + FN)$$
$$F_1 = 2PR/(P + R) \tag{4}$$

Let O be the 52 gold-operation cases, $A \subseteq O$ those returning a usable, non-unknown role, and $C \subseteq A$ those with the correct role. We report

$$\text{Coverage}_O = |A|/|O|,\ \text{Acc}_A = |C|/|A|$$
$$\text{Acc}_O = |C|/|O| \tag{5}$$

These are gold-operation-conditioned measures. Conditional role accuracy excludes false-positive operations on negative controls and must be read alongside presence errors. Lexical lookup produces presence flags but no usable roles; its conditional role accuracy is undefined, not zero. Table IV separates presence flags, usable role outputs, and correct roles. Historical presence scores for AST and full inference remain coupled to known-role outputs: they are not an independently evaluated presence-detection channel or a fair standalone detection ranking against an inventory-only baseline.

The operation non-answer rate is 1-Coverage_O; it is not the planner's abstention rate. Exact operation/role accuracy uses all 68 scored cases, requiring correct presence and, for gold operations, correct role and operation labels. Protocol accuracy uses the eight cases with known gold context. Error count is the number of cases with at least one scored mismatch, not the number of unsafe recommendations.

VII. LIMITATIONS AND THREATS TO VALIDITY

Construct validity. Snippet-level summaries couple detection to a known-role result. They do not measure complete per-call extraction, planner abstention, or recommendation safety. Random-source tracking is a proxy for key-transport purpose; evidence strength is not calibrated confidence.

Internal validity. One author constructed the corpus and labels, and development had access to scenario families. The lexical baseline is weak; no external analyzer was executed. Output masking is not internal component ablation. Bounded-analysis limitations include reassignment, aliasing,

wrappers, cross-scope flow, and mixed-role aggregation. Tests do not establish soundness.

External validity. Quantitative evidence is synthetic Python RSA only. It cannot establish Go accuracy, real-repository performance, other primitive/protocol coverage, or production dependency behavior. Existing repository scans lack the independent gold labels needed for accuracy claims. The nominal test partition is not a repository holdout.

Statistical conclusion validity. Confidence intervals, significance tests, independent agreement results, and population sampling are absent. Scenario correlation and purposive construction prevent treating snippets as independent natural-code draws. Perfect conditional accuracy on 40 selected answers must not be advertised as general reliability; two of three post-hoc probes already produced wrong role answers (Table V).

Security and deployment validity. Advisory planning does not prove confidentiality, unforgeability, or secure composition. Digest-contract tests provide no PQC interoperability evidence. Application remains disabled, public deployment is unverified, and faculty/security approval and release tagging are incomplete. Concrete integrations must check current standards and implementation errata.

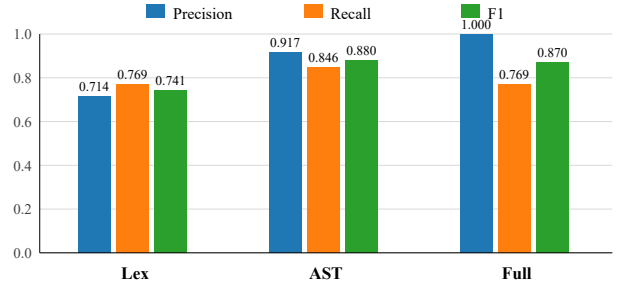
VIII. PILOT RESULTS AND ERROR ANALYSIS

A. Detection and role outcomes

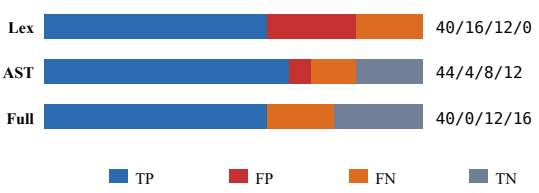
Tables III and IV report completed aggregate results. Values are rounded to three decimals; artifacts retain six decimals and raw predictions.

Table III. Historical presence-positive accounting on 68 cases. AST/full positives require a known role; lexical positives do not. These coupled scores must not be interpreted as an independent detection comparison.

(a) Presence-positive accounting



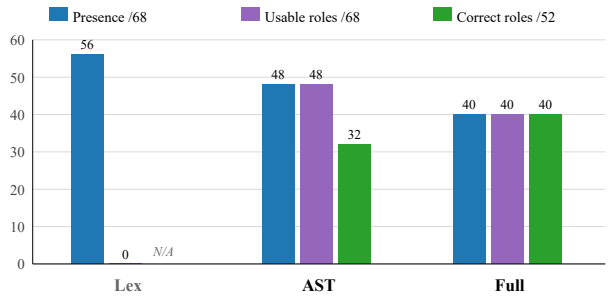
(b) Outcomes; exact tuple is TP/FP/FN/TN



Lex=lexical lookup; AST=method-name matching; Full=identity/context-aware resolver.

Table IV. Separate presence flags and usable role answers. AST's 48 role outputs include four false positives; 44 concern gold operations. Gold-operation role coverage is 44/52 for AST and 40/52 for full inference; end-to-end role accuracy is 32/52 and 40/52 respectively. Lexical lookup has no role-resolution capability.

Graphical view: output counts



Tabular view: exact counts and conditional accuracy

Configuration	Presence positives /68	Usable roles /68	Correct roles /52	Accuracy on answers
Lexical RSA lookup	56	0	N/A	N/A
AST method matching	48	48	32	32/44 = 0.727
Full resolver	40	40	40	40/40 = 1.000

The AST baseline detects four more gold operations than the full resolver but also produces four false positives. Its 0.880 F1 exceeds the full resolver's 0.870. However, it returns 32 correct roles among 44 answered gold operations, versus 40 among 40 for the full resolver. End-to-end role accuracy is consequently 40/52 = 0.769 for the full resolver and 32/52 = 0.615 for AST matching. These are descriptive differences, not statistically established effects.

Importantly, AST matching correctly resolves all 12 operations designated unsupported by the full resolver. Its remaining role errors comprise 12 key-transport cases labeled encryption, and its eight missed gold operations are JWT signatures. Thus stronger identity constraints sacrifice coverage on cases where simpler matching succeeds; the pilot does not establish that every resolver non-answer is beneficial or necessary.

AST succeeds on those 12 cases because it does not check which object receives the call. The same property produces its four false positives on negative controls, so its success there shows that a method name exists, not that the operation is RSA.

All 12 full-resolver errors belong to the unsupported-valid-operation category. Non-answer rate is 12/52 = 23.1% among gold operations. Across 68 scored cases, 28 predictions are operation-negative: 12 missed operations and 16 correctly rejected negative controls. Calling all 28 “abstentions on operations” would conflate outcomes. Exact operation/role accuracy is 56/68 = 82.4%, combining 40 correctly resolved operations and 16 correctly rejected negatives.

B. Sensitivity to output information

Masking session-secret information changes 12 correct key-transport roles to encryption. Detection counts remain unchanged while end-to-end role accuracy falls from 40/52 to 28/52. Detection metrics therefore cannot reveal the role distinction erased by the mask. This does not measure how a separately implemented analyzer without secret tracking would behave.

The protocol mask preserves operation and role predictions but changes eight correct JWT contexts to unknown. Context accuracy falls from 8/8 to 0/8 and error cases rise from 12 to 20. This demonstrates the accounting value of reporting context separately, with no evidence about TLS, SSH, certificates, or other contexts.

C. Post-hoc diagnostic probes

Three additional syntactically valid snippets were analyzed after the pilot was frozen, separately from the frozen 72-case corpus. Their intended purposes were declared by the corpus author, not independently annotated. They test named failure modes rather than estimate prevalence, advice correctness, or security.

Table V. Supplemental counterexamples; no pilot denominator or headline score is changed. Snippets and predictions are retained in `PQMIGRATE_PILOT_DIAGNOSTIC_PROBES.json`.

Diagnostic	Declared operation role	Resolver result
JWT encode with tracked RSA key and PS256	Signature	Unknown
RSA encrypts random challenge bytes	Encryption, not key transport	Key transport
RSA encrypts random padding bytes	Encryption, not key transport	Key transport

These outcomes expose a concrete signature-coverage gap and false-positive purpose inference from random-byte provenance. They invalidate an interpretation that the proxy itself proves transported key material.

D. Failures and unmeasured outcomes

Malformed inputs constitute 4/72 = 5.6% of the corpus. This is a designed-case fraction, not an estimated parser-failure rate on natural code. Diagnostics remain outside the principal score denominator. The current error taxonomy distinguishes presence, role, operation-label, and protocol mismatches; it does not establish every unsupported case's root cause independently.

No bootstrap intervals, independent-annotation agreement, deployment success rates, or false-recommendation rates have been completed. Forty correct roles among 40 answers do not establish zero future error, and zero detection false positives do not establish safe migration advice. A naive case-level interval would not remedy the authored corpus and correlated scenarios.

As a descriptive caution only, if 40 answers were independent identically distributed draws from a specified population, zero observed errors would yield an approximate 95% upper error bound of 3/40 = 7.5% by the rule of three [23] (exact one-sided binomial bound: 7.22%). Those sampling assumptions do not hold here, so neither value is a confidence bound for PQMigrate's real-code error rate.

IX. IMPLEMENTATION ASSURANCE AND REPRODUCIBILITY

A. Completed engineering evidence

The D0-D20 integration gate passed twice from a clean detached checkout of the same implementation commit. Preserved logs record 97 Python tests, 23 legacy scanner checks, five Maven tests, and a successful React/Vite build, together with later milestone scripts. These are software checks, not additional independently labeled research cases.

The preview/verification workflow demonstrates a narrow MD5-to-SHA-256 expression transformation. It binds a preview to a source hash, materializes a disposable copy, checks Python syntax and expected AST structure, and verifies that original source remains unchanged. This does not establish application-level semantic equivalence. The temporary filesystem tree is not a container, virtual machine, or full security sandbox.

The two-party laboratory evaluates a controlled digest-negotiation contract with agreement, fallback, no-common-mode, and tampered-digest scenarios. It does not implement ML-KEM, ML-DSA, or a standardized PQC hybrid protocol. These checks demonstrate the assurance workflow, not successful PQC migration.

Deployment evidence identifies a locally validated WSL CLI, loopback dashboard, local static-site preview, backend tests, and frontend build. There is no public hosting or production-deployment claim. Release tagging and faculty/security review remain pending. Dependency-audit findings require triage before release readiness.

B. Source and artifact identity

The source repository is available at the project URL [24]. The evaluated implementation commit is

`bcfb5e40fbc8eb74e1c919114f54031d03ab44e9`

The SHA-256 of `benchmarks/pqmigratebench_v0_1.jsonl` is

`293fee315c4fbcb95f0d071615ce54a7ac6c9b8b89783a832832c15e3697603b`

The full gate is `bash scripts/validate_d0_d20.sh`; the focused baseline gate is `bash scripts/run_d18_baselines.sh`. The D20 manifest records the source commit and milestone-artifact hashes. Complete logs, artifacts, and checksums were preserved separately under `pqmigrate-release-evidence/bcfb5e4/run1/` and `run2/` in the local environment. This avoids changing the validated source commit to hold regenerated evidence. Local preservation is verified; an immutable public evidence deposit has not been established.

Reproduction requires the documented Python dependencies and Tree-sitter grammar, Java/Maven, and Node/npm dependencies. Historical runs reused the configured project environment. Clean source checkout is supported; a hermetic build from a new machine is not established. Later documentation revisions do not change the identity of the evaluated implementation.

X. Discussion and expanded limitations

The synthetic pilot diagnoses what the bounded resolver can and cannot express. It cannot support claims of superiority over external tools, real-code accuracy, reduced migration effort or safer recommendations. Better detection F1 can coexist with worse role assignment; conversely, selective non-answers can inflate correctness among answered cases while leaving useful operations unresolved. The historical table must be interpreted with both the task and denominator visible.

Internal and construct validity. A single author selected examples, defined labels and implemented the analyzer. Repeated scenario templates and a deliberately supported/unsupported mixture can favor the implementation. Snippet presence was historically coupled to a known role for some systems. Call-level operation presence, application role and planner advice are distinct constructs. Masking outputs is not a feature-removal experiment. Neither non-execution nor passing syntax/schema checks establishes semantic correctness. The random-byte key-transport proxy is specifically insufficient under the reviewer SOP.

External validity. The completed numerical evidence is Python/RSA-specific and synthetic. It does not establish Go performance, wrapper resolution, package-boundary behavior, protocol coverage or operational deployment characteristics. The 500 candidate-selected anchors focus on scanner-discoverable locations; even complete annotation of them cannot reveal all missed operations. The separate ordinary-source/independent-search packet is needed to study omissions. Whole-file Semgrep scans provide more context than isolated snippets but do not solve sampling bias or establish whole-repository coverage.

Statistical validity. Correlated examples, author selection and few repository families limit inference. No significance test, real-code confidence interval or inter-annotator agreement result is available. Once units and gold are locked, report raw denominators, label prevalence, undefined metrics and repository dependence alongside any interval. A planned repository bootstrap is not automatically reliable with few independent clusters; resampling must preserve paired systems and repository families. There is no statistical basis for selectively publishing only improvements over the historical table.

Reproducibility and standards drift. Pin the code, corpus, rule files, mapping, dependencies and standards access date. Mutable product documentation is evidence of documented capabilities at access time, not of a frozen tool's measured behavior. The inspected source and corpus are available at [24]; externally stored historical gate logs remain a verification gap. The IEEE-formatted bibliography identifies primary sources and versions; the accompanying audit distinguishes standards, author manuscripts, documentation, repository evidence and publisher metadata.

Planning and security validity. A resolved role is necessary evidence for some migration advice, not sufficient evidence of a correct redesign. KEM selection does not itself settle key confirmation, authentication, serialization, failure handling or protocol integration [1], [4]. ML-DSA identifiers in a protocol specification do not prove deployment readiness [7]. No cryptographic security proof, verified PQC transformation, production interoperability measurement or user study is contributed here. Separate proposed cryptography investigations contribute no result to this paper.

XI. Ongoing real-code evaluation protocol

The 19-23 October 2026 annotation window is an organizational schedule, not evidence of completion. Reviewer eligibility requires not having built the scanner/resolver or seen predictions for the target repositories. Both reviewers independently receive the same 500 anchors, not disjoint halves. No A/B submissions were inspected for this revision or the baseline work. Reviewer interaction about individual cases remains prohibited until both packets are submitted and locked.

A separate unscored calibration batch of approximately 25 anchors must precede production work. It is not the production packet's first 25 rows. The organizer freezes an agreed feasible list of production candidate IDs before review; unassigned rows remain unreviewed and cannot be selected afterward based on favorable labels. Each reviewer works on a private copy with identifying columns and row order preserved.

XI. Ongoing real-code evaluation protocol (continued)

At each pinned repository commit, reviewers inspect the whole containing function and follow imports, declarations, assignments, wrappers and callers as needed. The hash refers to the entire source file. Source is read and searched; target builds, installers and code are not executed. A proven source-level crypto call or wrapper is an operation even in tests; an import or familiar method name alone is not.

The SOP labels presence as yes/no/uncertain. Operation values are sign, verify, encrypt, decrypt, key_establish, hash, mac, kdf, key_generate, other_crypto, none and uncertain. Role values are signature, key_transport, key_establishment, encryption, hash, mac, kdf, key_management, other, none and uncertain. Unknown receivers or opaque boundaries require an uncertainty reason, rather than a guessed crypto identity. A non-crypto call uses no/none/none. Key creation maps to key_generate/key_management. Application evidence is required for key transport; encrypting random bytes alone is insufficient.

For multiple distinct operations at an anchor, presence is yes, operation and role are uncertain, and a MULTIPLE note identifies exact spans for adjudicator expansion. A wrong-line anchor is needs_context rather than silently relocated. Every completed row needs context and decisive source evidence. Operational definitions for setup calls, streaming stages and one-to-many alignments must be frozen before scored mapping; unresolved questions have been flagged to Ratnesh in repository issue 1 [24]. His instruction is to use the SOP taxonomy.

The study lead receives and locks both original files privately before comparing them. Preserve original disagreements and rationales, then adjudicate; do not replace original labels when computing agreement. Report raw agreement and Cohen's kappa [25] per applicable nominal field with prevalence, completed-pair counts, missing/unreviewed rows and any undefined denominator. No agreement has yet been measured. Calibration, assigned production rows and adjudicated expansions must be traceable as different stages.

After the first wave, the separate 240-anchor independent-search/ordinary-source packet is intended to investigate missed operations. Its design, frozen units, discovery effort and review coverage must be documented before estimating recall or TPR. The candidate-selected packet alone cannot support these claims. Repository families and near duplicates should remain in the same split; final sample sizes must be reported as observed rather than treated as achieved targets.

Baseline and failure accounting. The Semgrep comparison uses the same pinned Python/Go source revisions, published custom rules and the SOP vocabulary [26]. Preserve native findings and spans, raw stdout/stderr, exact versions, input/rule hashes, runtime limits and failure ledgers. Parse failures, timeouts, crashes, skipped files and unavailable sources must remain visible. A partial scan may contain findings and still be incomplete; neither a failure nor a no-match is automatically a gold negative. Duplicate/nested findings require a prespecified alignment policy, not favorable post hoc matching.

Do not tune rules from A/B submissions. Gold must be locked before accuracy calculations, and reporting must include unfavorable results and missingness. The current revision contains no Semgrep accuracy estimate, no real-code precision/recall/TPR, and no inter-annotator agreement. Future table changes require an auditable, comparable experiment and a transparent version note rather than a rule that only better outcomes are published.

XII. Conclusion

PQMigrate provides an inspectable bounded resolver within an advisory-planning prototype. Its completed evidence is a synthetic Python RSA pilot with explicit counterexamples and measurement limits. Independent real-code annotation, operation-unit adjudication and an unscored reproducible Semgrep baseline are ongoing. Stronger accuracy, safety, interoperability and transformation claims require separate evidence.

References

[1] National Institute of Standards and Technology, Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final

[2] National Institute of Standards and Technology, Module-Lattice-Based Digital Signature Standard, FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204. [Online]. Available: https://csrc.nist.gov/pubs/fips/204/final

[3] D. Ott, C. Peikert, and other workshop participants, "Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility," Computing Community Consortium workshop report, arXiv:1909.07353, 2019, doi: 10.48550/arXiv.1909.07353. [Online]. Available: https://arxiv.org/abs/1909.07353

[4] K. F. Hasan et al., "A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies," IEEE Access, 2024, doi: 10.1109/ACCESS.2024.3360412; author version arXiv:2307.06520v2. [Online]. Available: https://arxiv.org/abs/2307.06520v2

[5] National Institute of Standards and Technology, Stateless Hash-Based Digital Signature Standard, FIPS 205, Aug. 2024, doi: 10.6028/NIST.FIPS.205. [Online]. Available: https://csrc.nist.gov/pubs/fips/205/final

[6] National Institute of Standards and Technology, "Post-Quantum Cryptography," project status. Accessed: Oct. 21, 2026. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography

[7] M. Prorock and O. Steele, "ML-DSA for JSON Object Signing and Encryption (JOSE) and CBOR Object Signing and Encryption (COSE)," RFC 9964, May 2026, doi: 10.17487/RFC9964. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9964.html

[8] NIST National Cybersecurity Center of Excellence, "Migration to Post-Quantum Cryptography." Accessed: Oct. 21, 2026. [Online]. Available: https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc

[9] M. Moffie et al., "Cryptoscope: Analyzing cryptographic usages in modern software," arXiv:2503.19531v1, Mar. 2025, doi: 10.48550/arXiv.2503.19531. [Online]. Available: https://arxiv.org/abs/2503.19531

[10] S. Krüger et al., "CogniCrypt: Supporting Developers in using Cryptography," author manuscript, 2017. [Online]. Available: https://www.eclipse.org/cognicrypt/resources/publications/knr%2B17cognicrypt.pdf

[11] S. Krüger, J. Späth, K. Ali, E. Bodden, and M. Mezini, "CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs," in ECOOP, LIPIcs, vol. 109, 2018, pp. 10:1-10:27, doi: 10.4230/LIPIcs.ECOOP.2018.10. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2018.10

[12] S. Rahaman et al., "CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects," arXiv:1806.06881v5, 2019, doi: 10.48550/arXiv.1806.06881. [Online]. Available: https://arxiv.org/abs/1806.06881v5

[13] S. Afrose, S. Rahaman, and D. D. Yao, "CryptoAPI-Bench: A Comprehensive Benchmark on Java Cryptographic API Misuses," in Proc. IEEE SecDev, 2019, pp. 49-61; benchmark artifact. [Online]. Available: https://github.com/CryptoAPI-Bench/CryptoAPI-Bench

[14] GitHub, "Use of a broken or weak cryptographic algorithm," CodeQL Python query documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://codeql.github.com/codeql-query-help/python/py-weak-cryptographic-algorithm/

[15] CBOMkit contributors, "Sonar Cryptography Plugin," source and documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://github.com/cbomkit/sonar-cryptography

[16] CBOMkit contributors, "CBOMkit," source and documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://github.com/cbomkit/cbomkit

[17] OWASP Foundation, "Cryptography Bill of Materials (CBOM)," CycloneDX documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://cyclonedx.org/capabilities/cbom/

[18] OASIS, Static Analysis Results Interchange Format (SARIF) Version 2.1.0 Plus Errata 01, OASIS Standard, Aug. 2023. [Online]. Available: https://docs.oasis-open.org/sarif/sarif/v2.1.0/errata01/os/sarif-v2.1.0-errata01-os-complete.html

[19] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," arXiv:1705.08500v2, 2017, doi: 10.48550/arXiv.1705.08500. [Online]. Available: https://arxiv.org/abs/1705.08500v2

[20] C. K. Chow, "On Optimum Recognition Error and Reject Tradeoff," IEEE Trans. Inf. Theory, vol. 16, no. 1, pp. 41-46, Jan. 1970, doi: 10.1109/TIT.1970.1054406. [Online]. Available: https://research.ibm.com/publications/on-optimum-recognition-error-and-reject-tradeoff

[21] R. El-Yaniv and Y. Wiener, "On the Foundations of Noise-free Selective Classification," J. Mach. Learn. Res., vol. 11, no. 53, pp. 1605-1641, 2010. [Online]. Available: https://www.jmlr.org/papers/v11/el-yaniv10a.html

[22] M. Jones, "JSON Web Algorithms (JWA)," RFC 7518, May 2015, doi: 10.17487/RFC7518. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7518.html

[23] J. A. Hanley and A. Lippman-Hand, "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators," JAMA, vol. 249, no. 13, pp. 1743-1745, Apr. 1983, doi: 10.1001/jama.1983.03330370053031. [Online]. Available: https://jamanetwork.com/journals/jama/article-abstract/385438

[24] PQMigrate project, "PQMigrate source code and evaluation artifacts," Git repository, evaluated revision bcfb5e40fbc8eb74e1c919114f54031d03ab44e9, 2026. [Online]. Available: https://github.com/Ratneshp1122/PQMigrate/tree/bcfb5e40fbc8eb74e1c919114f54031d03ab44e9

[25] J. Cohen, "A Coefficient of Agreement for Nominal Scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37-46, Apr. 1960, doi: 10.1177/001316446002000104. [Online]. Available: https://journals.sagepub.com/doi/abs/10.1177/001316446002000104

[26] Semgrep, "Run rules," documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://docs.semgrep.dev/running-rules